Computer Graphics (CACS305)

Unit - 03: Two Dimensional Geometric Transformations

Compiled by Shishir Ghimire

Credit Hours: 3hrs

TABLE OF CONTENTS

Unit 3: Two-Dimensional Geometric Transformations (5 Hrs.)

3.1 Two-Dimensional translation, Rotation, Scaling, Reflection and Shearing

3.2 Homogeneous Coordinate and 2D Composite Transformations. Transformation between Co-ordinate Systems.

3.3 Two Dimensional Viewing: Viewing pipeline, Window to viewport coordinate transformation

3.4 Clipping: Point, Lines(Cohen Sutherland line clipping, Liang-Barsky Line Clipping) , Polygon Clipping(Sutherland Hodgeman polygon clipping)

2-Dimensional Transformation

3.1

INTRODUCTION

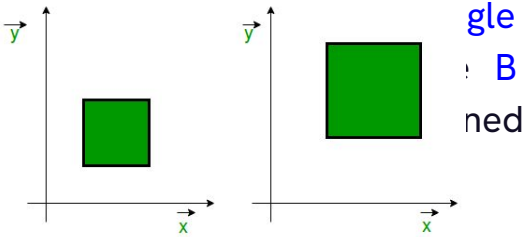
Transformation: Changing coordinate description of an object is called transformation.

Types:

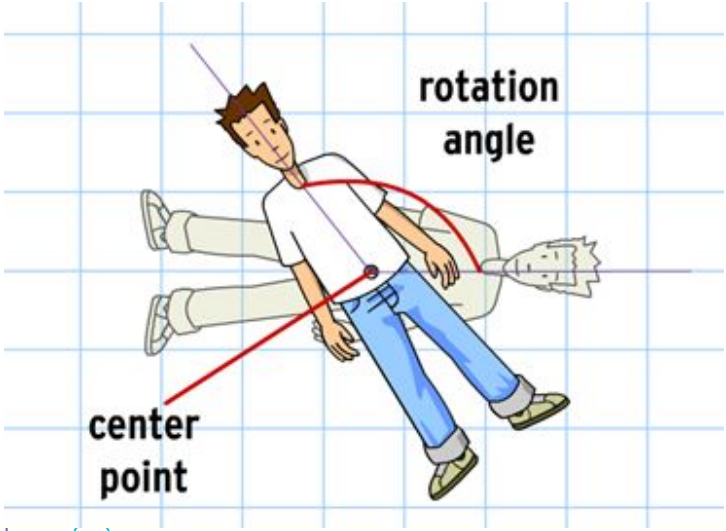
- Rigid body transformation (transformation without change in shape.):
  - Translation
  - Rotation
  - Reflection
- Non rigid body transformation (transformation with change in shape.):
  - Scaling
  - Shearing

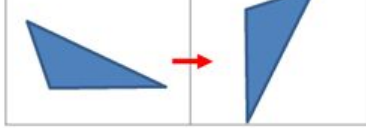
Two essential aspects of transformation are given below:

1. Each transformation is a **single entity.** It can be denoted by a unique name or symbol.
2. It is possible to **combine** two transformations, gle **transformation** is obtained, e.g., A is a transforn B transformation performs scaling. The combination of ned by concatenation property.



INTRODUCTION



	enlarged reduced flipped **rotated**
	enlarged reduced **flipped** **rotated**
	enlarged reduced flipped **rotated**
	enlarged reduced **flipped** **rotated**
	enlarged **reduced** flipped rotated
	**enlarged** reduced flipped rotated
	enlarged reduced flipped **rotated**

2D Transformation:

- When a transformation takes place on a 2D plane, it is called 2D transformation.
- The transformations are used directly by application programs and within many graphics subroutines in application programs.

Types of Transformations:

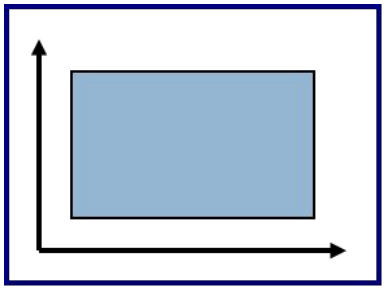
The **three** basic transformations are

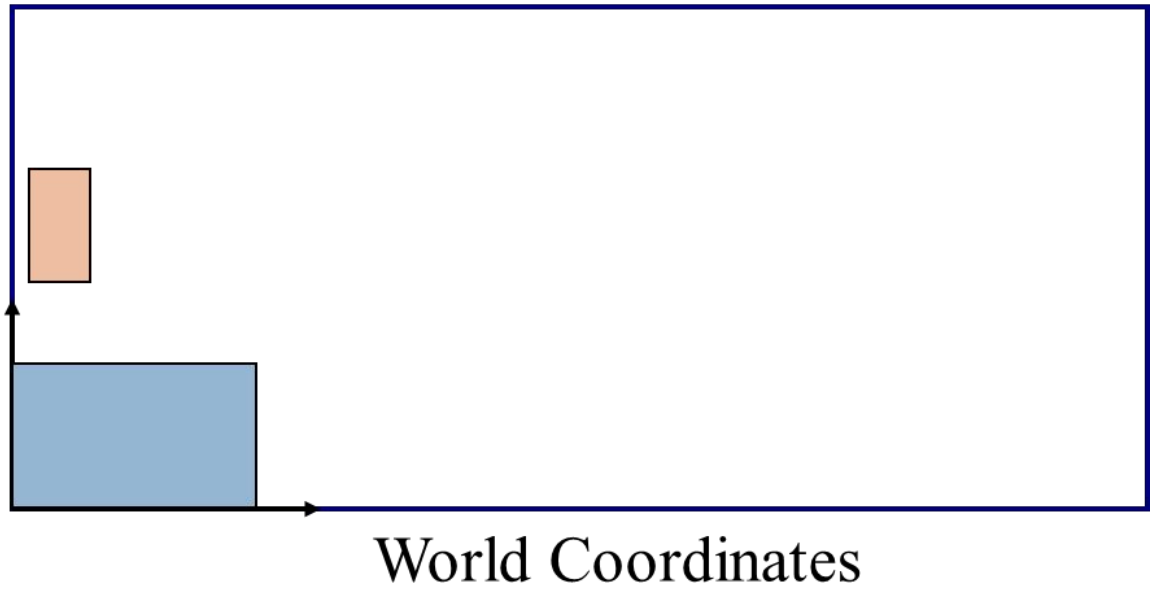
1. **Translation**
2. **Rotation**
3. **Scaling**

Other transformation includes **reflection** and **shear**.

2-Dimensional Translation, Rotation, Scaling

2D Transformation > Basics :





2D Transformation > Basics :

- Represent a 2D Transformation by a Matrix:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

- Apply the Transformation to a Point:

$$x' = ax + by$$
$$y' = cx + dy$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Transformation Matrix

Point

2D Translation:

Repositioning of object along a straight-line path from one coordinate location to another is called translation.

- Translation is performed on a point by adding translation distance to its coordinate so as to generate a new coordinate position.
- Let p(x, y) be translated to $p'(x', y')$ by adding translation distance t_x and t_y to the respective coordinates in x & y direction. Then,

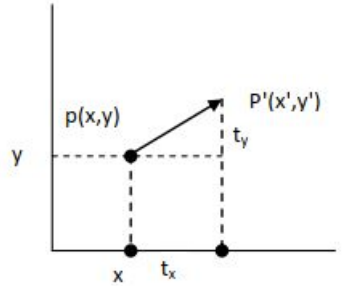
$$x' = x + t_x$$
$$y' = y + t_y$$

- The translation distance pair (t_x, t_y) is known as translation vector or shift vector. We can express translation equations as matrix representations as:

$$P = \begin{bmatrix} x \\ y \end{bmatrix} \quad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} \quad T = \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

$$\therefore P' = P + T$$

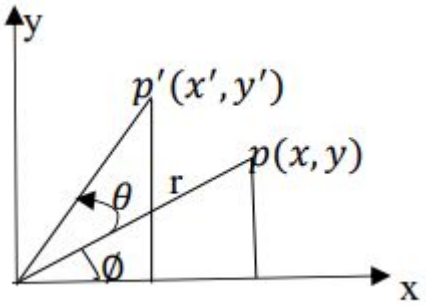
$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$



2D Rotation:

(Reposition) Changing the coordinate position along a circular path is called rotation.

- 2D rotation is applied to re-position the object along a circular path in XY-plane. Rotation is generated by specifying rotation angle (θ) and pivot point (rotation point) about which the object is to be rotated.
- Rotation can be made by angle θ either clockwise or anticlockwise direction.
  - The positive θ rotates object in anti-clockwise direction and the negative value of θ rotates the object in clockwise direction.
- A line perpendicular to rotating plane and passing through pivot point is called axis of rotation.



2D Rotation:

Rotation through Origin (0,0):

- Let P(x, y) is a point in XY-plane which is to be rotated with angle θ about origin to new point $p'(x', y')$. Also let OP = r (As in figure below) is constant distance from origin. Let r makes angle θ with positive X-direction as shown in figure:
- When OP is rotated through angle and taking origin as pivot point for rotation, OP' makes angle $\theta+\varnothing$ with X-axis.

Here,

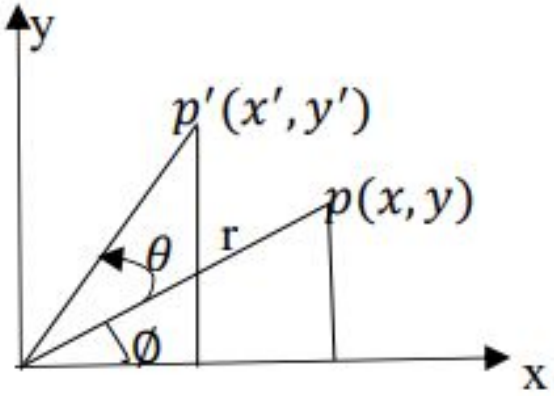
$$x' = r\cos(\varnothing + \theta)$$
$$= r\cos\varnothing\cos\theta - r\sin\varnothing\sin\theta$$

But $x = r\cos\varnothing$ & $y = r\sin\varnothing$

$$\therefore \boldsymbol{x' = x\cos\theta - y\sin\theta} \quad \text{..... (i)}$$

Similarly,

$$\therefore \boldsymbol{y' = x\sin\theta + y\cos\theta} \quad \text{..... (ii)}$$



which are equation for rotation of (x, y) with angle θ and taking pivot as origin.

2D Rotation:

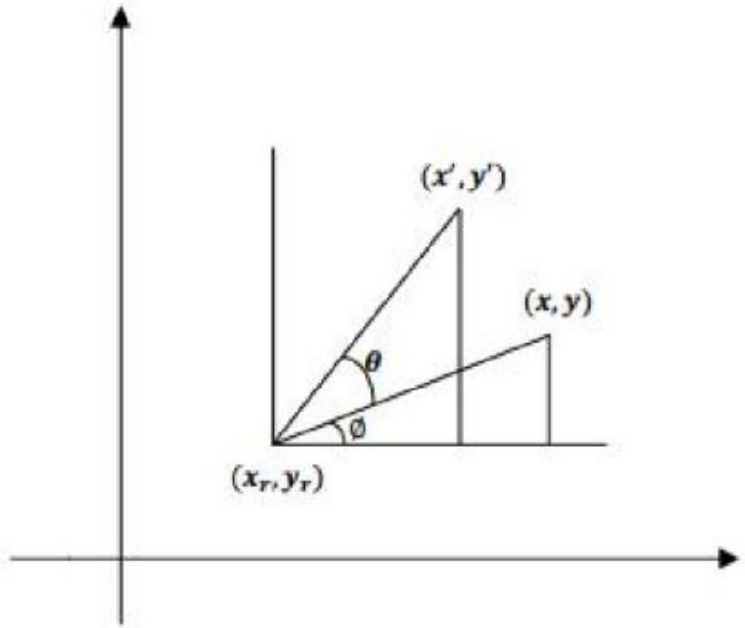
In matrix form

$$P' = R.P$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} cos\theta & -sin\theta \\ sin\theta & cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotation from any arbitrary pivot point (x_r, y_r):

- Let Q(x_r, y_r) is pivot point for rotation.
- P(x, y) is co-ordinate of point to be rotated by angle θ.
- Let ϕ is the angle made by QP with X-direction. θ.



Then angle made by QP' with X-direction is $\theta + \phi$

Hence,

$$\cos(\phi+\theta) = (x'-x_r)/r$$

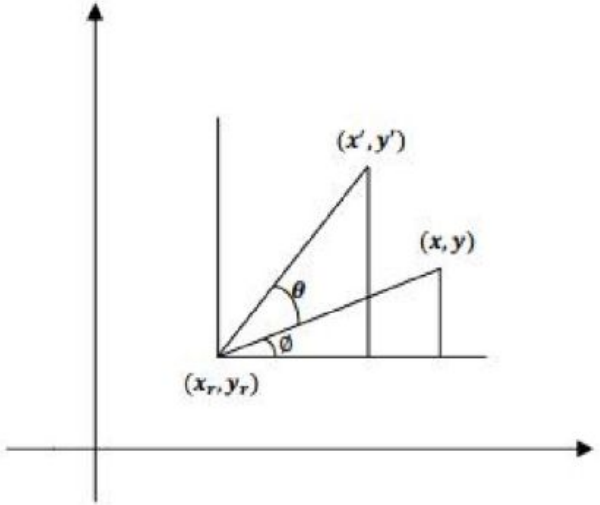
or $$r\cos(\phi+\theta) = (x'-x_r)$$

or $$x'-x_r = r\cos\phi\cos\theta - r\sin\phi\sin\theta$$

since $r\cos\phi = x - x_r, r\sin\phi = y - y_r$

$$x' = x_r + (x-x_r)\cos\theta - (y-y_r)\sin\theta \quad \text{..... (1)}$$

2D Rotation:



Similarly,

$\sin(\emptyset + \theta) = (y' - y_r)/r$
$or, r\,sin(\emptyset + \theta) = (y' - y_r)$
$or, (y' - y_r) = rsin\emptyset cos\theta + rcos\emptyset sin\theta$

Since, $rcos\emptyset = (x - x_r), rsin\emptyset = (y - y_r)$

$\therefore\ \boldsymbol{y' = y_r + (x - x_r)sin\theta + (y - y_r)cos\theta}$ ..... (ii)

These equations (i) and (ii) are the equations for rotation of a point (x, y) with angle θ taking pivot point(x_r, y_r).

2D Scaling:

A scaling transformation alters the size of the object. This operation can be carried out by multiplying the co-ordinate values (x, y) of each vertex by scaling factor s_x and s_y to produce transformed co-ordinates (x', y').

i.e. $x' = x.s_x$ and $y' = y.s_y$

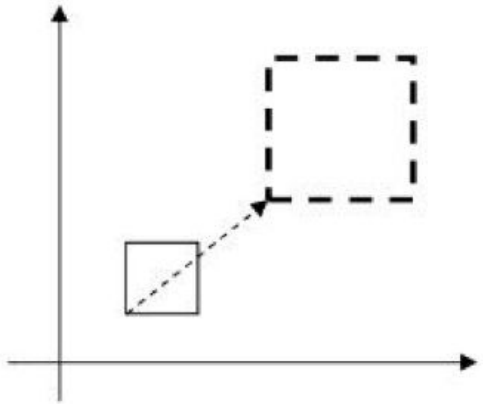
Scaling Factor:

- Scaling factor s_x scales object in x- direction and s_y scales in y- direction. If the scaling factor is less than 1, the size of object is decreased and if it is greater than 1 the size of object is increased. The scaling factor = 1 for both direction does not change the size of the object.
    - If both scaling factors have same value then the scaling is known as uniform scaling.
    - If the value of s_x and s_y are different, then the scaling is known as differential scaling. The differential scaling is mostly used in the graphical package to change the shape of the object.

2D Scaling:

The matrix equation for scaling is:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$ i.e. P' = S.P



Homogeneous Coordinates, Reflection & Shear Transform

3.2

Matrix Representation & Homogeneous Coordinate

Issue: Many graphics applications involve sequences of geometric transformations. Example: An animation might require an object to be translated and rotated at each increment of the motion.

To combine these three transformations into a single transformation, homogeneous coordinates are used. In homogeneous coordinate system, two-dimensional coordinate positions (x, y) are represented by triple-coordinates.

How such transformation sequences can be efficiently processed using matrix notation?

- The homogeneous co-ordinate system provides a uniform framework for handling different geometric transformations, simply as multiplication of matrices.
- To perform more than one transformation at a time, homogeneous coordinates are used. They reduce unwanted calculations, intermediate steps, saves time and memory and produce a sequence of transformations.
- To express any two-dimensional transformation as a matrix multiplication, we represent each Cartesian coordinate position *(x, y)* with the homogeneous coordinate triple (x_h, y_h, h), where $x = x_h/h$, $y = y_h/h$.(h is 1 usually for 2D case).
  By using this homogeneous co-ordinate system a 2D point would be (x, y, 1).

Matrix Representation & Homogeneous

For Translation: $T(t_x, t_y)$

$$P' = T.P$$

Where, $P' = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}, T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$ & $P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

By which we can get,

$x' = x + t_x$

$y' = y + t_y$

- For **2D Rotation**: R(θ)
  - **About origin** the homogeneous matrix equation will be

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Which gives the

$x' = x\cos\theta - y\sin\theta$ ---------- 1

$y' = x\sin\theta + y\cos\theta$ ---------- 2 which are equation for rotation of (x, y) with angle θ and taking pivot as origin.

- Rotation **about an arbitrary fixed pivot point (x_r,y_r)**

For a fixed pivot point rotation, we can apply composite transformation as Translate fixed point (x_r, y_r) to the co-ordinate origin by T(-x_r,-y_r). Rotate with angle θ → R(θ) .Translate back to original position by T(x_r,y_r).This composite transformation is represented as:

P' = T(x_r, y_r).R(θ).T(-x_r, -y_r). This can be represented in matrix equation using homogeneous co-ordinate system as,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & x_r \\ 0 & 1 & y_r \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -x_r \\ 0 & 1 & -y_r \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Expanding this equation we get

$x' = x_r + (x - x_r)\cos\theta - (y - y_r)\sin\theta$ ..... (1)

$y' = y_r + (x - x_r)\sin\theta + (y - y_r)\cos\theta$ ..... (2)

These are actually the equations for rotation of (x, y) from the pivot point (x_r, y_r).

Matrix Representation & Homogeneous Coordinate

For Scaling with scaling factors (s_x, s_y)

$$P' = S.P$$

Where, $P' = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$, $S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$ & $P = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

This gives the equations,

$x' = x.s_x$ & $y' = y.s_y$

Q. What is the basic purpose of composite transformation?

→ *The basic purpose of composing transformation is to gain efficiency by applying a single composed transformation to a point, rather than applying a series of transformation, one after another.*

Composite Transformations:

Composite 2D translation

If two successive translation vector (t_{x1}, t_{y1}) & (t_{x2}, t_{y2}) is applied on position $p(x, y)$, then translated point is given by,

$P' = T_{(t_{x2},t_{y2})} . T_{(t_{x1},t_{y1})} . P$

$$\therefore \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_{x2} \\ 0 & 1 & t_{y2} \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 & 0 & t_{x1} \\ 0 & 1 & t_{y1} \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Which gives,

$x' = x + t_{x1} + t_{x2}$

$y' = y + t_{y1} + t_{y2}$

#Q. Prove that two successive translations are additive.

Proof:

If two successive translation vector (t_{x1}, t_{y1}) & (t_{x2}, t_{y2}) is applied to coordinate position P, the final transformed location P' is calculated with the following composite transformation as,

$T = T_{(t_{x2},t_{y2})} . T_{(t_{x1},t_{y1})}$

$$= \begin{bmatrix} 1 & 0 & t_{x2} \\ 0 & 1 & t_{y2} \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 & 0 & t_{x1} \\ 0 & 1 & t_{y1} \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_{x1} + t_{x2} \\ 0 & 1 & t_{y1} + t_{y2} \\ 0 & 0 & 1 \end{bmatrix}$$

Hence, $T_{(t_{x2},t_{y2})} . T_{(t_{x1},t_{y1})} = T_{(t_{x1}+t_{x2},t_{y1}+t_{y2})}$ which demonstrates that two successive translations are additive.

Composite Transformations:

2D composite rotation

$$P' = (R(\theta_2)).(R(\theta_1)).P$$
$$= \mathrm{R}\,(\theta_1 + \theta_2).P$$

#Q. Prove that two successive rotation are additive.

Proof:

Let P be the point anticlockwise rotated by angle θ_1 to point P' and again let P' be rotated by angle θ_2 to point P", then the combined transformation can be calculated with the following composite matrix as:

$$T = R(\theta_2).R(\theta_1)$$
$$= \begin{bmatrix} cos\theta_2 & -sin\theta_2 & 0 \\ sin\theta_2 & cos\theta_2 & 0 \\ 0 & 0 & 1 \end{bmatrix}. = \begin{bmatrix} cos\theta_1 & -sin\theta_1 & 0 \\ sin\theta_1 & cos\theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} cos\theta_2 * cos\theta_1 - sin\theta_2 * sin\theta_1 & -cos\theta_2 * sin\theta_1 - sin\theta_2 * cos\theta_1 & 0 \\ sin\theta_2 * cos\theta_1 + cos\theta_2 * sin\theta_1 & -sin\theta_2 * sin\theta_1 + cos\theta_2 * cos\theta_1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$
$$= \begin{bmatrix} \cos(\theta_1 + \theta_2) & -sin(\theta_1 + \theta_2) & 0 \\ sin(\theta_1 + \theta_2) & cos(\theta_1 + \theta_2) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

i.e. $R(\theta_2).R(\theta_1) = R(\theta_1 + \theta_2)$ which demonstrates that two successive rotations are additive.

Composite Transformations:

2D composite Scaling

$$P' = S_{(s_{x2},s_{y2})} \cdot S_{(s_{x1},s_{y1})} \cdot P$$

$$P' = \begin{bmatrix} s_{x2} & 0 & 0 \\ 0 & s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_{x1} & 0 & 0 \\ 0 & s_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

#Q. Prove that two successive scaling are multiplicative.

Proof:

Let point P is first scaled with scaling factors s_{x1}, s_{y1} to P' and again let P' be scaled by scaling factors s_{x2}, s_{y2} to point P", then the combined transformation can be calculated with the following composite matrix

$$T = S_{(s_{x2},s_{y2})} \cdot S_{(s_{x1},s_{y1})}$$

$$= \begin{bmatrix} s_{x2} & 0 & 0 \\ 0 & s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} s_{x1} & 0 & 0 \\ 0 & s_{y1} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} s_{x1}s_{x2} & 0 & 0 \\ 0 & s_{y1}s_{y2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

i.e. $S_{(s_{x2},s_{y2})} \cdot S_{(s_{x1},s_{y1})} = S_{(s_{x1}s_{x2},s_{y1}s_{y2})}$ which demonstrates that two successive scaling are multiplicative.

General 2D pivot rotation

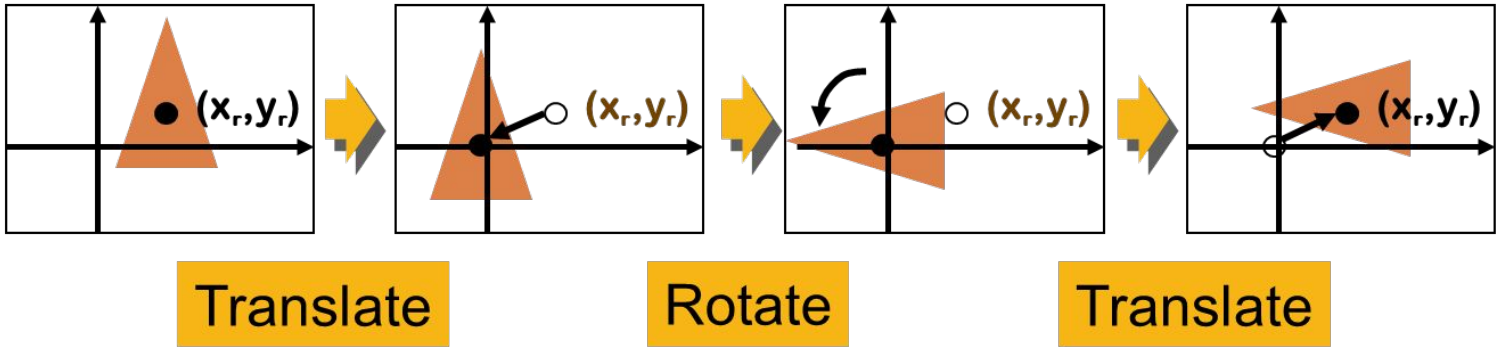
- Suppose the pivot point is located at (x_r, y_r).
- To rotate about arbitrary point, we have to perform the following transformation:
  a) Translate the object so that pivot point position is moved to coordinate origin.
  b) Rotate the object about coordinate origin.
  c) Translate the object so that pivot point is returned to original position.

Matrix representation:

$$\begin{bmatrix} 1 & 0 & x_r \\ 0 & 1 & y_r \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} cos\theta & -sin\theta & 0 \\ sin\theta & cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 & 0 & -x_r \\ 0 & 1 & -y_r \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} cos\theta & -sin\theta & x_r(1-cos\theta) + y_r sin\theta \\ sin\theta & cos\theta & y_r(1-cos\theta) - x_r sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

General 2D pivot rotation



$$T(x_r, y_r) \cdot R(\theta) \cdot T(-x_r, -y_r) = R(x_r, y_r, \theta)$$

$$\begin{bmatrix} 1 & 0 & x_r \\ 0 & 1 & y_r \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -x_r \\ 0 & 1 & -y_r \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & x_r(1-\cos\theta) + y_r \sin\theta \\ \sin\theta & \cos\theta & y_r(1-\cos\theta) - x_r \sin\theta \\ 0 & 0 & 1 \end{bmatrix}$$

General 2D fixed point scaling

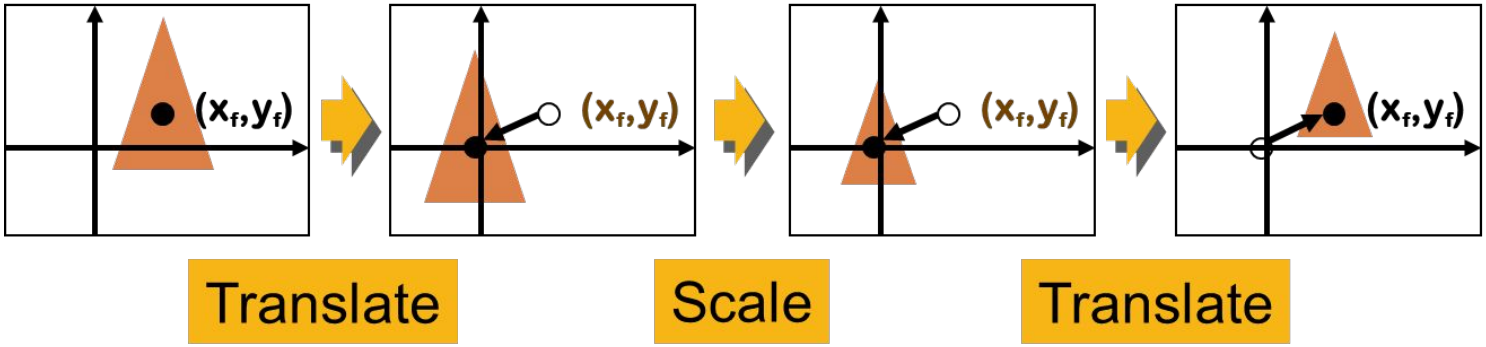
- Suppose the fixed point is located at (x_f, y_f).
- To scale about arbitrary fixed point, we have to perform the following transformation:
  d) Translate the object so that fixed point coincide with the coordinate origin.
  e) Scale the object with respect to coordinate origin.
  f) Translate the object to its original position.

Matrix representation:

$$\begin{bmatrix} 1 & 0 & x_f \\ 0 & 1 & y_f \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} . \begin{bmatrix} 1 & 0 & -x_f \\ 0 & 1 & -y_f \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} s_x & 0 & x_f(1-s_x) \\ 0 & s_y & y_f(1-s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

General 2D fixed point scaling



$$T(x_f,y_f)\cdot S(s_x,s_y)\cdot T(-x_f,-y_f)=S(x_f,y_f,s_x,s_y)$$

$$\begin{bmatrix}1 & 0 & x_f\\ 0 & 1 & y_f\\ 0 & 0 & 1\end{bmatrix}\cdot\begin{bmatrix}s_x & 0 & 0\\ 0 & s_y & 0\\ 0 & 0 & 1\end{bmatrix}\cdot\begin{bmatrix}1 & 0 & -x_f\\ 0 & 1 & -y_f\\ 0 & 0 & 1\end{bmatrix}=\begin{bmatrix}s_x & 0 & x_f(1-s_x)\\ 0 & s_y & y_f(1-s_y)\\ 0 & 0 & 1\end{bmatrix}$$

Reflection:

A reflection is a transformation that produces a mirror image of an object. In 2D-transformation, reflection is generated relative to an axis of reflection. The reflection of an object to an relative axis of reflection, is same as 180° rotation about the reflection axis.

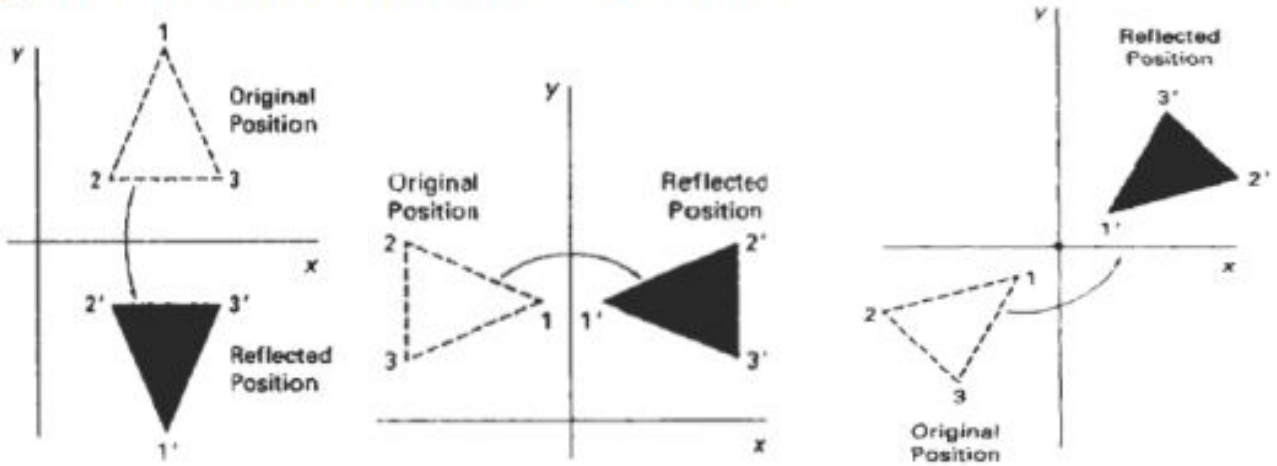


Fig: reflection of an object about an x-axis, y-axis and axis perpendicular to xy-plane (passing through origin) respectively.

Reflection:

Reflection about x-axis (y=0)

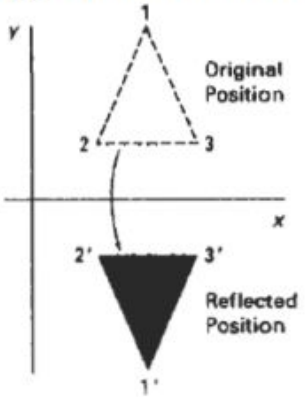
The reflection of a point P(x, y) on x-axis, changes the y-coordinate sign i.e. P(x, y) changes to P'(x, -y).

In matrix form,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$P' = R_{fx}.P$

R_{fx}= Reflection matrix about x-axis.



Reflection:

Reflection about y-axis (x=0)

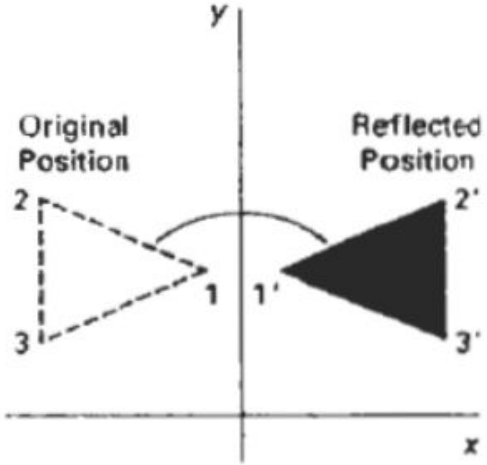
The reflection of a point P(x, y) on y-axis changes the sign of x-coordinate i.e. P(x, y) changes to P'(-x, y).

In matrix form,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$P' = R_{fy}.P$

R_{fy}= Reflection matrix about y-axis.



Reflection:

Reflection about origin: The reflection on the line perpendicular to xy-plane and passing through flips x and y co-ordinates both. So sign of x and y co-ordinate value changes. The equivalent matrix equation for the point is:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Reflection about y=mx+c

Perform the following transformation:

a) Translate the line so that it passes through origin.
b) Rotate the line so that it coincides with any coordinate axis.
c) Reflect object about that axis.
d) Perform reverse rotation.
e) Perform reverse translation so that line is placed to its original position.

Shearing (Parallelogram Effect):

A transformation that distorts the shape of an object such that the transformed shape appears as if the object is composed of internal layers and these layers are caused to slide over is shearing.

X-direction Shear:

An X-direction shear relative to x-axis is produced with transformation matrix equation.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Which transforms,

$$x' = x + sh_x . y$$

$$y' = y$$

Y-direction shear:

A Y-direction shear relative to y-axis is produced by following transformation equations.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Which transforms,

$$x' = x$$

$$y' = x . sh_y + y$$

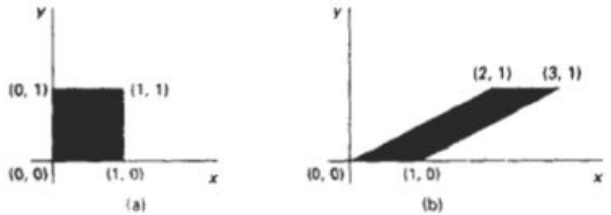
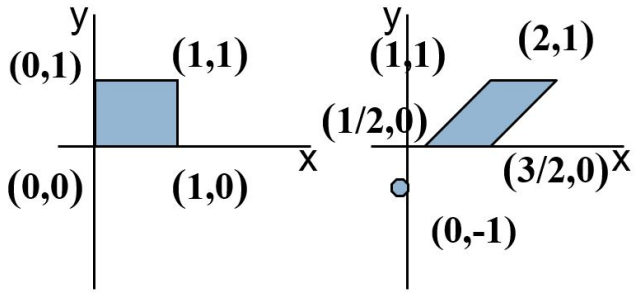


Fig: A unit square (a) is converted to a parallelogram (b) using the *x*-direction shear matrix with $sh_x = 2$.

Shearing:



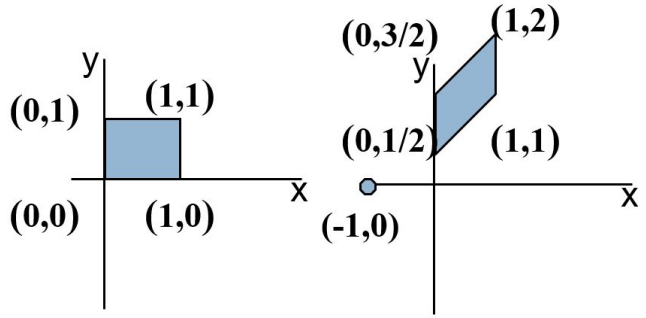
$(Sh_x = 1/2,\ y_{ref} = -1)$

X-direction shear relative to $y = y_{ref}$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & -sh_x.y_{ref} \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' = x + sh_x(y - y_{ref})$$

$$y' = y$$



$(Sh_y = 1/2,\ x_{ref} = -1)$

Y-direction shear relative to $x = x_{ref}$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ sh_y & 1 & -sh_y.x_{ref} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$$x' = x$$

$$y' = y + sh_y(x - x_{ref})$$

Two Dimensional Viewing

3.3

2D Viewing:

Definition: Two Dimensional viewing is the formal mechanism for **displaying views of a picture** on an output device.

- Typically, a **graphics package allows** a user to specify which part of a defined picture is to be displayed and where that part is to be placed on the display device.
- For a two dimensional picture, **a view is selected** by specifying a subarea of the total picture area. The picture parts within the selected areas are then mapped onto specified areas of the device coordinates.

Basic Terms:

Window: A world-coordinate area selected for display is called window.

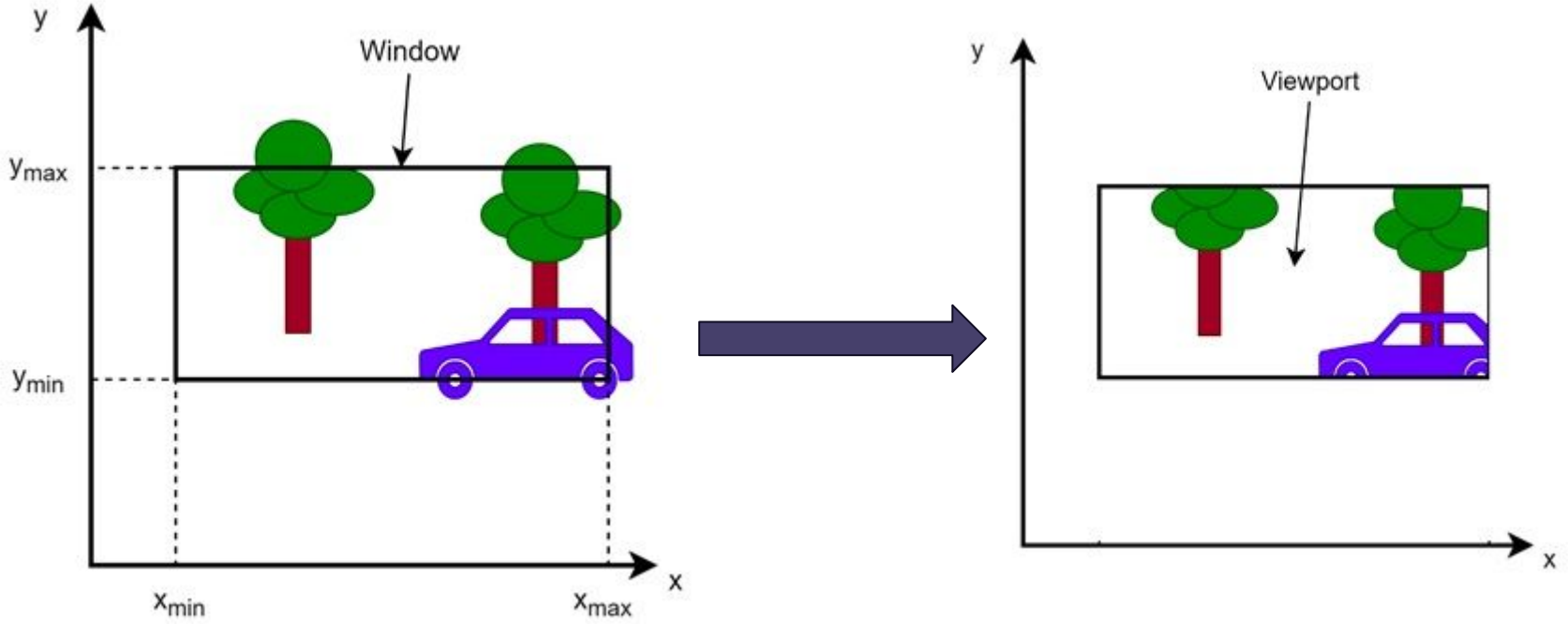
- The window defines what is to be viewed

Viewport: An area on a display device to which a window is mapped is called viewport.

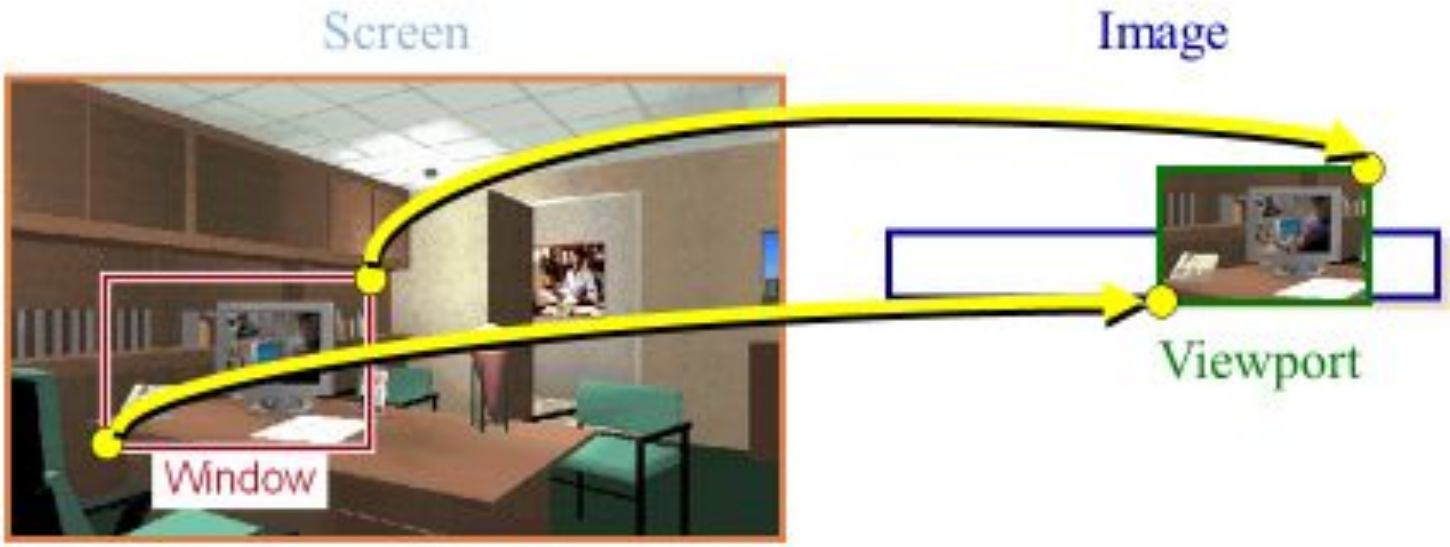
- The viewport defines where it is to be displayed.

- Window deals with object space whereas viewport deals with image space.

2D Viewing:



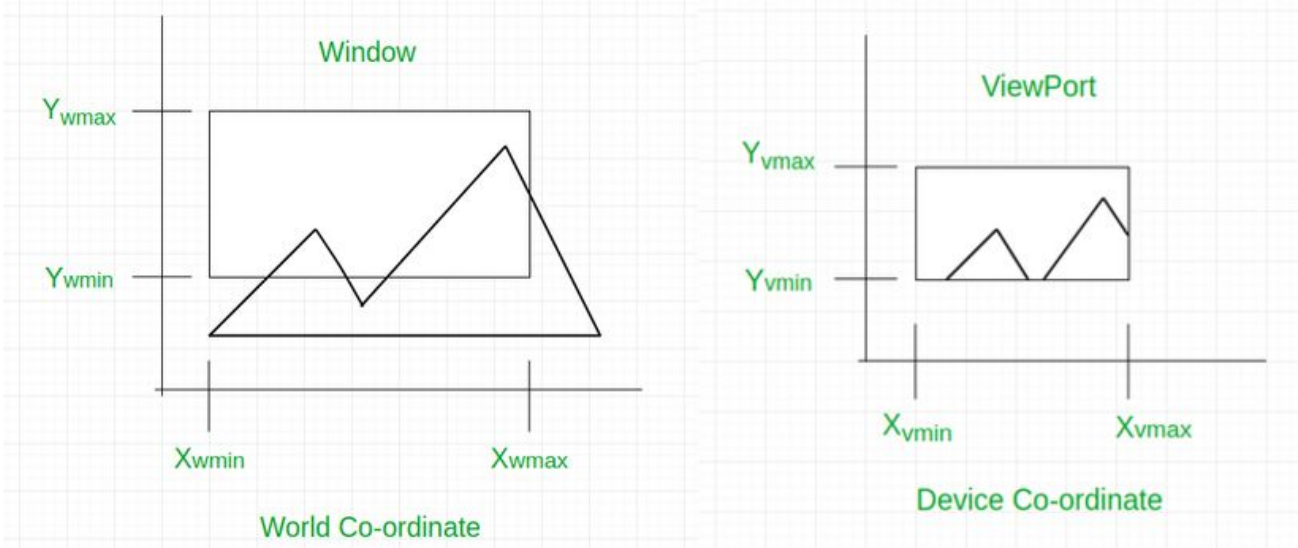
2D Viewing:



2D Viewing:

The process of mapping the world coordinate scene to device coordinate is called viewing transformation or windows to viewport transformation.

- The two dimensional viewing transformation is sometimes referred to as window to viewport transformation.



2D Viewing:

- Transformations from world to device coordinate involves **translation, rotation and scaling operations**, as well as procedure for deleting those parts of the picture that are outside the limits of selected display area i.e. **clipping**.
- To make the viewing process **independent** of the requirements of any output device, graphics systems convert object description to **normalized coordinates**

Viewing Transformation in Several Steps:

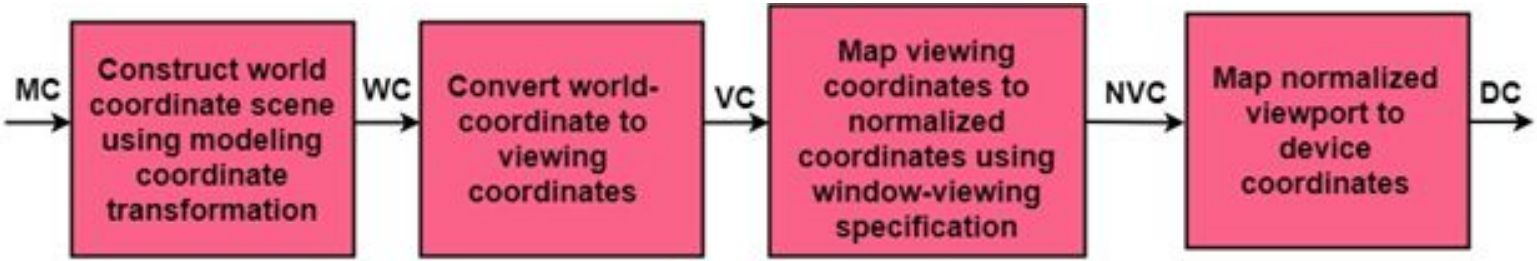


Fig: The two-dimensional viewing-transformation.

2D Viewing:

Viewing Transformation in Several Steps:

1. First, we construct the scene in **world coordinate** using the output primitives and attributes.
2. To obtain a particular orientation, we can set up a **2-D viewing coordinate system** in the window coordinate plane and define a window in viewing coordinates system.
3. Once the viewing frame is established, are then transform description in world coordinates to **viewing coordinates**.
4. Then, we define viewport in **normalized coordinates (range from 0 to 1)** and map the viewing coordinates description of the scene to normalized coordinates.
5. At the final step, all parts of the picture that (i.e., outside the viewport are dipped, and the contents are transferred to **device coordinates**).

2D Viewing:

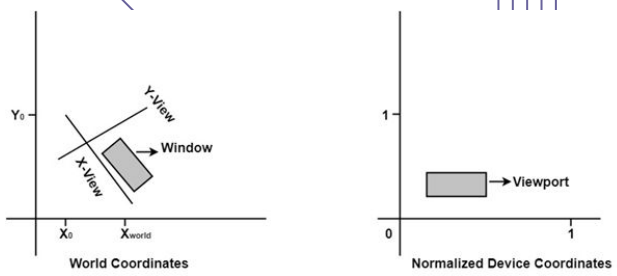


Fig:Setting up a rotated world window and corresponding normalized coordinate viewport.

Applications:

1. By changing the **position** of the view port, we can **view objects at different positions** on the display area of an output device.
2. By varying the **size of view ports**, we can change size of displayed objects.
3. **Zooming effects** can be obtained by successively mapping **different-sized** windows on a **fixed-sized view port**.
4. Panning effects **(Horizontal scrolling)** are produced by moving a fixed-sized window across the various objects in a scene.

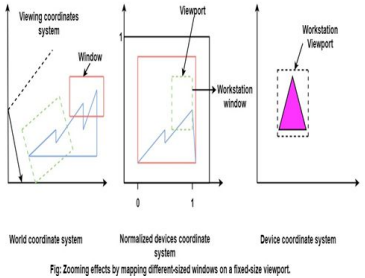


Fig: Zooming effects by mapping different-sized windows on a fixed-size viewport.

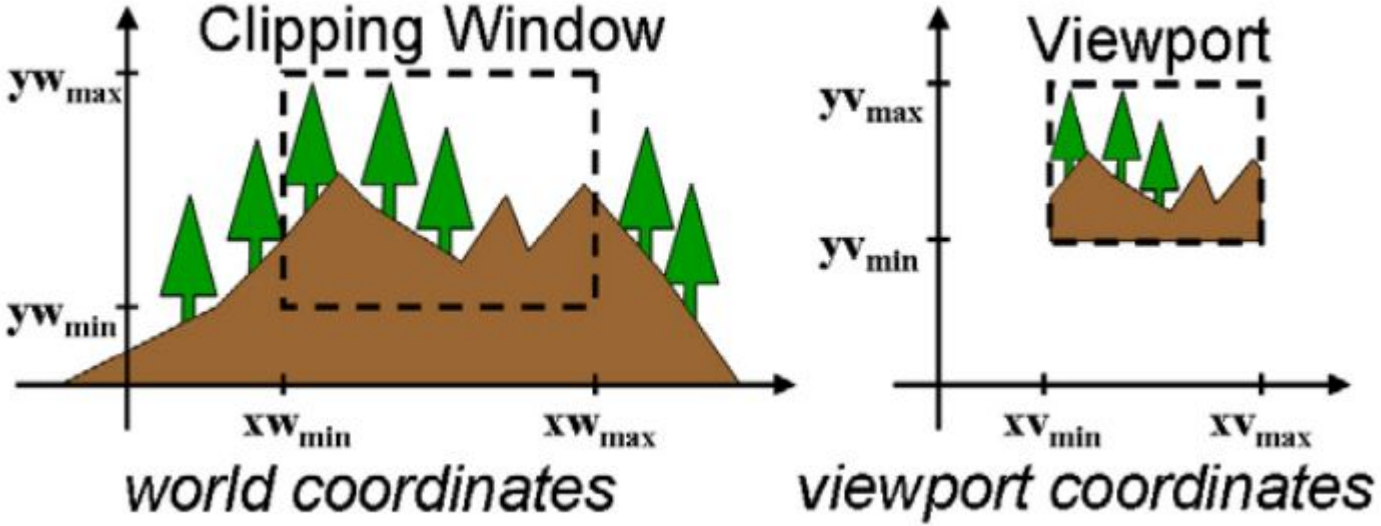
Window to Viewport Transformation:

Definition: The mapping of part of **world coordinate** scene to **device coordinate** is called viewing transformation or window-to-viewport transformation.

- Once, object description has been **transmitted** to the viewing reference frame, we choose the window extends in **viewing coordinates** and selects the **viewport limits** in normalized coordinates.
- Object descriptions are then **transferred** to normalized device coordinates:
  - We do this thing using a **transformation** that maintains the same relative placement of an object in normalized space as they had in viewing coordinates.
- If a coordinate position is at the **center of the viewing window**:
  - It will display at the center of the viewport.

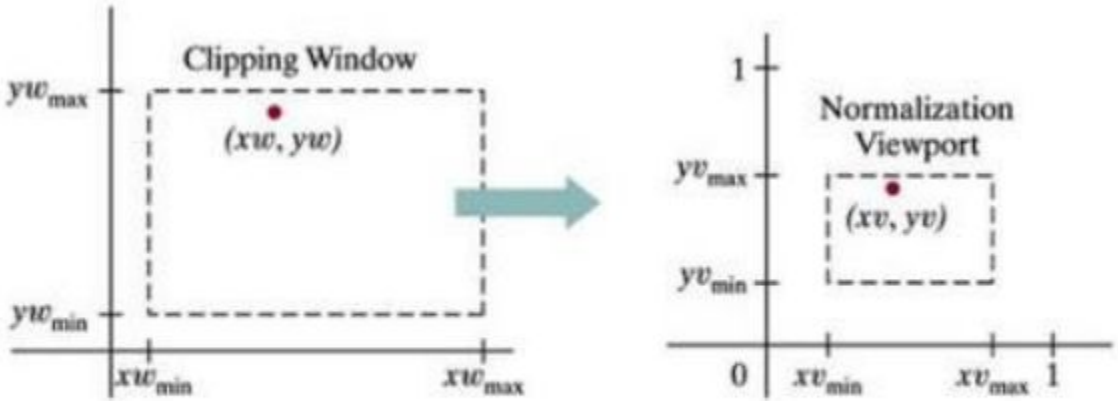
Window to Viewport Transformation:

Fig shows the window to viewport mapping. A point at position (xw, yw) in window mapped into position (xv, yv) in the associated viewport.



Window to Viewport Transformation:

Fig shows the window to viewport mapping. A point at position (xw, yw) in window mapped into position (xv, yv) in the associated viewport.



A point (xw, yw) in a world-coordinate clipping window is mapped to viewport coordinates (xv, yv), within a unit square, so that the relative positions of the two points in their respective rectangles are the same.

Window to Viewport Transformation:

To maintain the same relative placement in the viewport as in the window, we require that:

$$\frac{xv - xv_{min}}{xv_{max} - xv_{min}} = \frac{xw - xw_{min}}{xw_{max} - xw_{min}}$$

$$\frac{yv - yv_{min}}{yv_{max} - yv_{min}} = \frac{yw - yw_{min}}{yw_{max} - yw_{min}}$$

By solving these equations for the unknown viewport position (xv, yv) the following becomes true:

$$xv = s_x xw + t_x$$
$$yv = s_y yw + t_y$$

The scale factors (s_x, s_y) would be,

$$s_x = \frac{xv_{max} - xv_{min}}{xw_{max} - xw_{min}}$$

$$s_y = \frac{yv_{max} - yv_{min}}{yw_{max} - yw_{min}}$$

Window to Viewport Transformation:

And the translation factors (t_x, t_y) would be,

$$t_x = \frac{xw_{max}.xv_{min} - xw_{min}.xv_{max}}{xw_{max} - xw_{min}}$$

$$t_y = \frac{yw_{max}.yv_{min} - yw_{min}.yv_{max}}{yw_{max} - yw_{min}}$$

Transforming world coordinate to view coordinate can be obtained with the following sequence:

a) Scale (S) the clipping window to the size of viewport using fixed point position of (xw_{min}, yw_{min}).
b) Translate (T) (xw_{min}, yw_{min}) to (xv_{min}, yv_{min}).

(Translate the scaled window area to the position of the viewport.)

Window to Viewport Transformation:

Here,

$$S = \begin{bmatrix} s_x & 0 & xw_{min}(1-s_x) \\ 0 & s_y & yw_{min}(1-s_y) \\ 0 & 0 & 1 \end{bmatrix}$$

$$T = \begin{bmatrix} 1 & 0 & xv_{min} - xw_{min} \\ 0 & 1 & yv_{min} - yw_{min} \\ 0 & 0 & 1 \end{bmatrix}$$

Now, window- to – viewport transformation matrix is;

$$\boldsymbol{T_{wv} = T.S = \begin{bmatrix} s_x & 0 & t_x \\ 0 & s_y & t_y \\ 0 & 0 & 1 \end{bmatrix}}$$

Window to Viewport Transformation Numericals:

Q. Window port is given by (100, 100, 300, 300) and viewport is given by (50, 50, 150, 150). Convert the window port coordinate (200, 200) to the view port coordinate.

Window to Viewport Transformation Numericals:

Q. Window port is given by (100, 100, 300, 300) and viewport is given by (50, 50, 150, 150). Convert the window port coordinate (200, 200) to the view port coordinate.

Solution:

Here,

$(xw_{min}, yw_{min}) = (100,100)$
$(xw_{max}, yw_{max}) = (300,300)$
$(xv_{min}, yv_{min}) = (50,50)$
$(xv_{max}, yv_{max}) = (150,150)$
$(xw, yw) = (200,200)$

Then we have

$$s_x = \frac{xv_{max} - xv_{min}}{xw_{max} - xw_{min}} = \frac{150-50}{300-100} = 0.5$$

$$s_y = \frac{yv_{max} - yv_{min}}{yw_{max} - yw_{min}} = \frac{150-50}{300-100} = 0.5$$

$$t_x = \frac{xw_{max}.xv_{min} - xw_{min}.xv_{max}}{xw_{max} - xw_{min}} = \frac{300\times50 - 100\times150}{300-100} = 0$$

$$t_y = \frac{yw_{max}.yv_{min} - yw_{min}.yv_{max}}{yw_{max} - yw_{min}} = \frac{300\times50 - 100\times150}{300-100} = 0$$

The equation for mapping window coordinate to view port coordinate is given by,

$$xv = s_x xw + t_x$$
$$yv = s_y yw + t_y$$

Hence,

$$xv = 0.5 \times 200 + 0 = 100$$
$$yv = 0.5 \times 200 + 0 = 100$$

The transformed viewport coordinate is (100, 100).

Window to Viewport Transformation Numericals:

Q. Find the normalization transformation matrix for window to viewport which uses the rectangle whose lower left corner is at (2, 2) and upper right corner is at (6, 10) as a window and the viewport that has lower left corner at (0, 0) and upper right corner at (1, 1).

Window to Viewport Transformation Numericals:

Q. Find the normalization transformation matrix for window to viewport which uses the rectangle whose lower left corner is at (2, 2) and upper right corner is at (6, 10) as a window and the viewport that has lower left corner at (0, 0) and upper right corner at (1, 1).

Solution:

We have

$$s_x = \frac{xv_{max} - xv_{min}}{xw_{max} - xw_{min}} = \frac{1-0}{6-2} = 0.25$$

$$s_y = \frac{yv_{max} - yv_{min}}{yw_{max} - yw_{min}} = \frac{1-0}{10-2} = 0.125$$

$$t_x = \frac{xw_{max}.xv_{min} - xw_{min}.xv_{max}}{xw_{max} - xw_{min}} = \frac{6\times0 - 2\times1}{6-2} = -0.5$$

$$t_y = \frac{yw_{max}.yv_{min} - yw_{min}.yv_{max}}{yw_{max} - yw_{min}} = \frac{10\times0 - 2\times1}{10-2} = -0.25$$

The composite transformation matrix for transforming the window coordinate to viewport coordinate is given as

$$T = T_{(t_x,t_y)} S_{(s_x,s_y)}$$

$$= \begin{bmatrix} s_x & 0 & t_x \\ 0 & s_y & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.25 & 0 & -0.5 \\ 0 & 0.125 & -0.25 \\ 0 & 0 & 1 \end{bmatrix}$$

Clipping

3.4

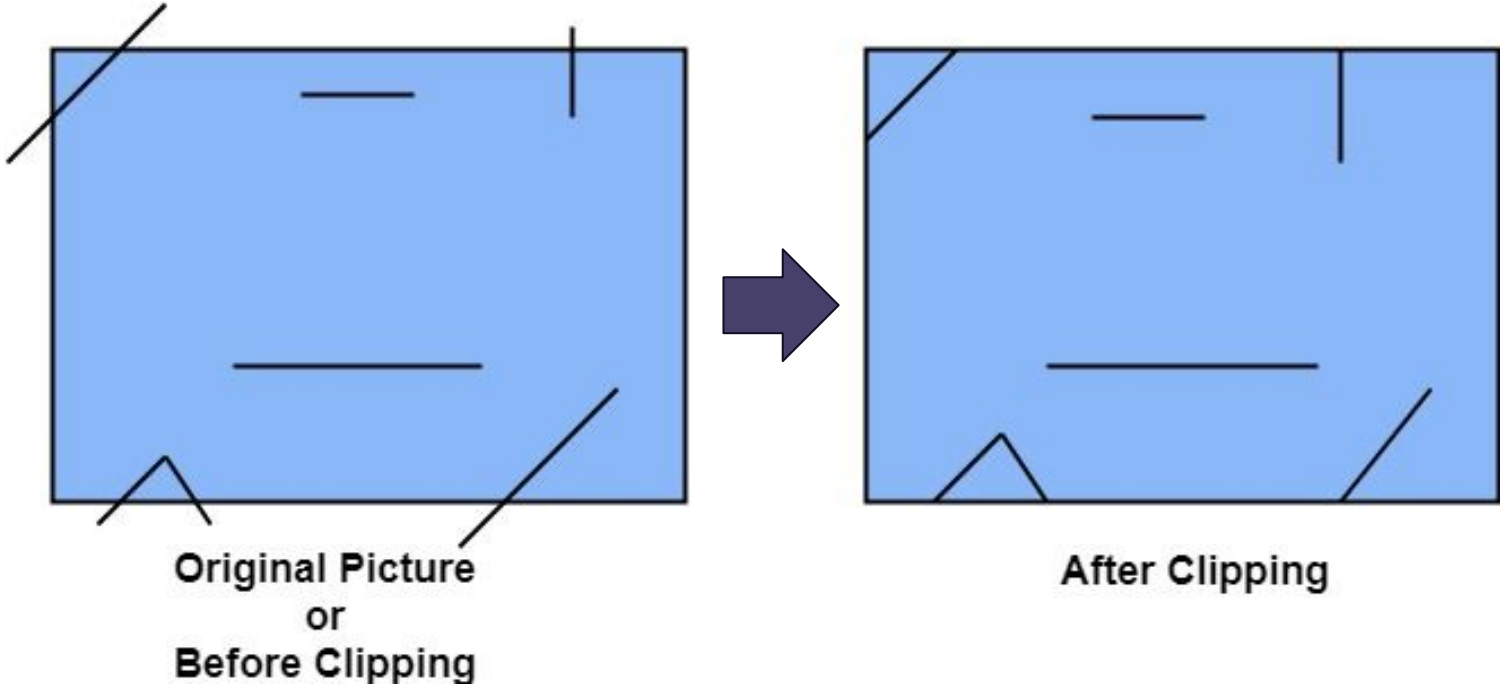
Clipping:

Definition:The procedure using which we can identify those portions of the graphics object (picture) that is either inside or outside a specified region or space is called clipping algorithm.

- The region against which an object is clipped is called a **clip window**.
- The process of **discarding** those parts of a picture which are outside of a specified region or window is called clipping.
- Clipping determines each element into the visible and invisible portion. Visible portion is selected. An invisible portion is discarded.
- Clipping is necessary to **remove those portions of the object which are not necessary** for further operations. It excludes unwanted graphics from the screen.
- So, there are **three cases**:
  - **Case-1:** The object may be **completely outside** the viewing area defined by the window port.
  - **Case-2:** The object may be seen **partially inside** in the window port.
  - **Case-3:** The object may be seen **completely inside** in the window port.
    - For case 1 & 2, clipping operation is necessary but not for case 3.

Clipping:

The window against which object is clipped called a clip window. It can be curved or rectangle in shape.



Clipping:

Applications of Clipping:

- Extracting part of a defined scene for viewing
- Identifying visible surfaces in three-dimensional views
- Antialiasing line segments or object boundaries
- Creating objects using solid-modeling procedures
- Displaying a multi-window environment
- Drawing and painting operations that allow parts of a picture to be selected for copying, moving, erasing, or duplicating.

Clipping:

Types of Clipping:

1. Point Clipping
2. Line Clipping
3. Area Clipping (Polygon)
4. Curve Clipping
5. Text Clipping
6. Exterior Clipping

Point Clipping:

Point Clipping is used to determining, whether the **point is inside the window or not**. For this following conditions are checked.

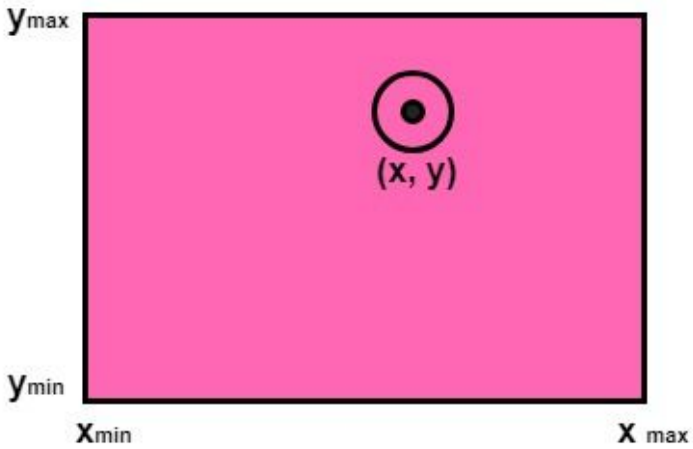
Assuming that the clip window is a rectangle in standard position, we save a point P=(x, y) for display if the following inequalities are satisfied:

$$xw_{min} \leq x \leq xw_{max}$$

$$yw_{min} \leq y \leq yw_{max}$$

Where the edges of the clip window $(xw_{min}, xw_{max}, yw_{min}, yw_{max})$ can be either the world-coordinate window boundaries or viewport boundaries. If any one of these four inequalities is not satisfied, the point is clipped.

Point Clipping:



The (x, y) is coordinate of the point. If anyone from the above inequalities is false, then the point will fall outside the window and will not be considered to be visible

Point Clipping:

Algorithm:

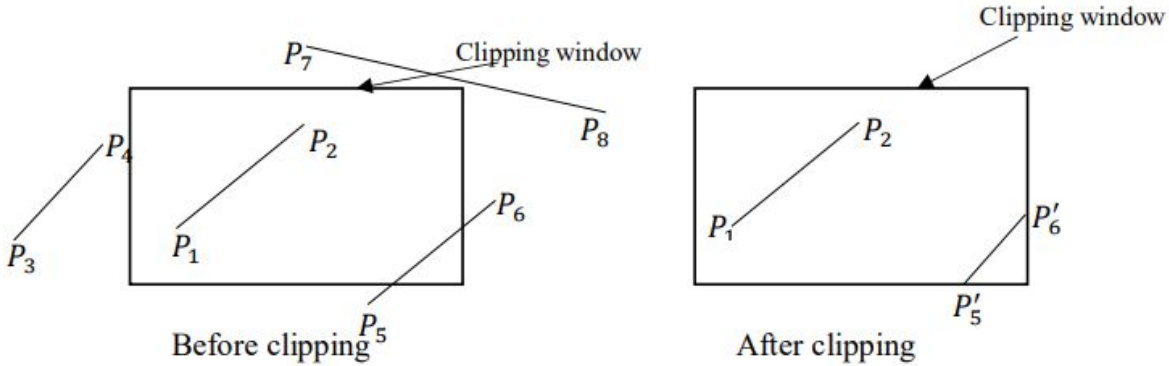
1. Get the minimum and maximum coordinates of the viewing plane.
2. Get the coordinates for a point.
3. Check whether given input lies between minimum and maximum coordinates of viewing plane.
4. If yes display the point which lies inside the region otherwise discard it.

Line Clipping:

In line clipping, a line or part of line is clipped if it is outside the window port.

There are **three possibilities** for the line (in a clipping procedure):

1. **Case-1:** Line can be **completely inside** the window (This line should be accepted).
2. **Case-2:** Line can be **completely outside** of the window (This line will be completely removed from the region).
3. **Case-3:** Line can be **partially inside** the window (We will find intersection point and draw only that portion of line that is inside region).



Cohen - Sutherland Line Clipping Algorithm

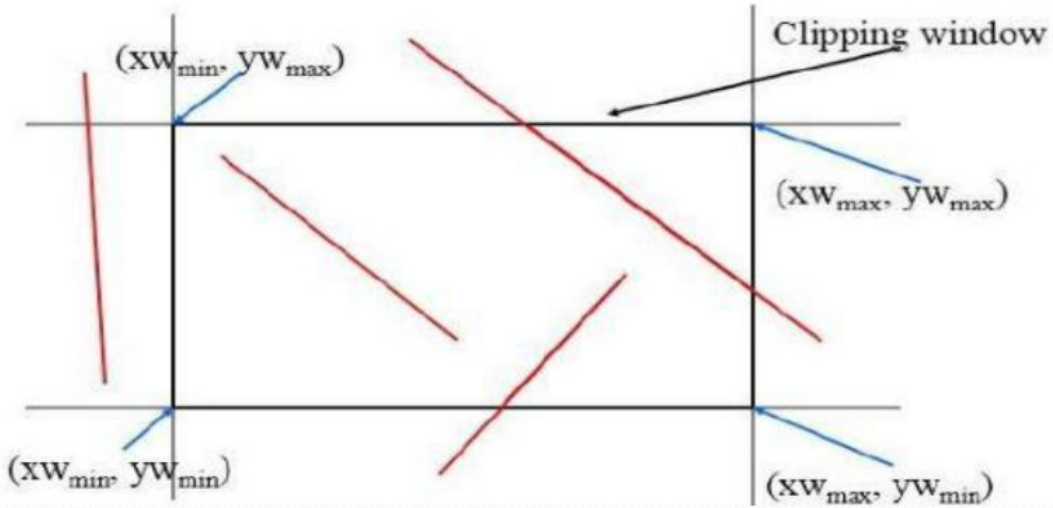
3.4.1

Cohen-Sutherland Line Clipping Algorithm:

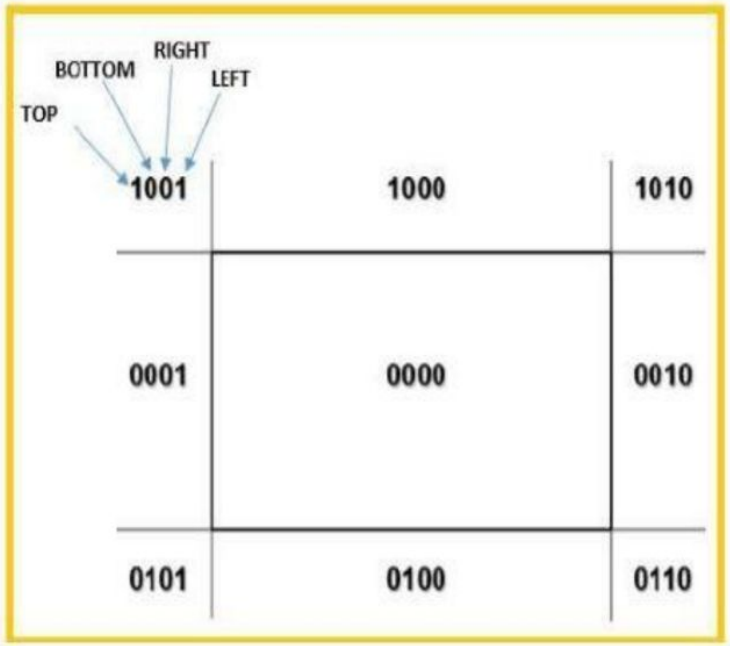
- This is one of the **oldest and most popular** line clipping algorithm.
- In this method, coordinate system is divided into **9 (nine) regions.** All regions have their **associated region codes.**
- Every **line endpoint** is assigned a four-digit binary code (region-code or out-codes). Each bit in the code is set to either a **true (1) or false (0).**
- And each region is assigned a 4-bit pattern.

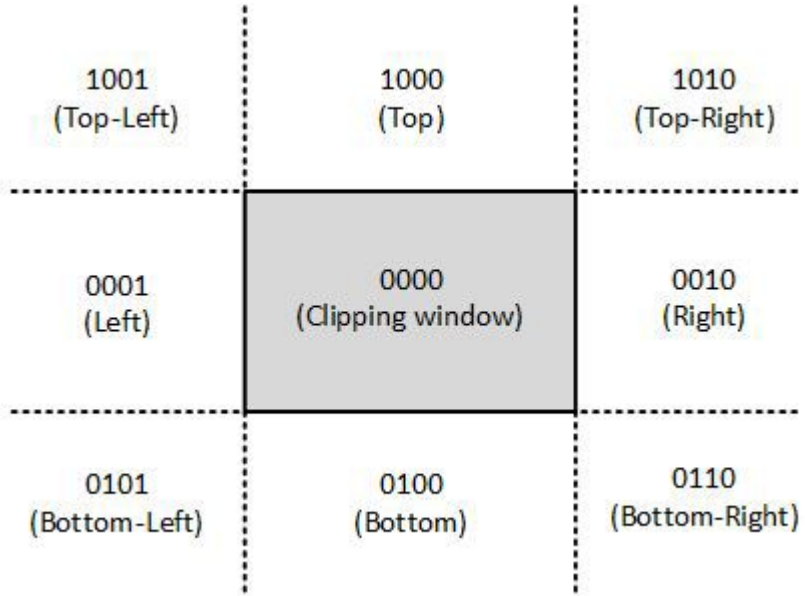
Cohen-Sutherland Line Clipping Algorithm:

This algorithm uses the clipping window as shown in the following figure. The minimum coordinate for the clipping region is (XW_{min}, YW_{min}) and the maximum coordinate for the clipping region is (XW_{max}, YW_{max}).



Cohen-Sutherland Line Clipping Algorithm:





Cohen-Sutherland Line Clipping Algorithm:

- In this algorithm, we are given 9 regions on the screen. Out of which **one region** is of the **window** and the **rest 8 regions** are around it given by **4 digit binary**. The division of the regions are based on (x_max, y_max) and (x_min, y_min).

- The central part is the **viewing region** or window, all the lines which lie within this region are **completely visible**. A region code is always assigned to the endpoints of the given line.

Cohen-Sutherland Line Clipping Algorithm:

- The numbers in above figure are called as region codes. Any point inside the **clipping window** has region code 0000.
- An outcode is computed for each of **two points of a line**.
- For any endpoint (x,y) of a line, the code can be determined **in which region** the endpoint lies.
- The outcode bits are set according to **following conditions:**
  - **L-First-bit** is set to 1, if point lies towards **left** of window.
  - **R-Second-bit** is set to 1, if point lies towards **right** of window.
  - **B-Third-bit** is set to 1, if point lies **below/bottom** of the window.
  - **T-Fourth-bit** is set to 1, if point lies **top/above** the window.

Cohen-Sutherland Line Clipping Algorithm:

Algorithm:

1. Given a line segment with **endpoints** P1= (x_1, y_1) and P2=(x_2, y_2).
2. Compute the **4-bit out-codes** for the two endpoints of the line segment.
3. If out-code of P1 **OR** out-code of P2 = 0000 (**P1 || P2 = 0000**) then
    a. Line lies completely inside the window, so draw the line segment.
4. Else If out-code of P1 **AND** out-code of P2 != 0000 (**P1 && P2 != 0000**) then
    a. Line lies completely outside the window, so discard the line segment.
5. Else If out-code of P1 **AND** out-code of P2 = 0000 (**P1 && P2 = 0000**) then
    a. Compute the **intersection of the line segment** with window boundary, and **discard** the portion of the segment that falls **completely outside the window**.
    b. Assign a **new 4-bit code** to the intersection and repeat until either 3 or 4 above is satisfied.

Cohen-Sutherland Line Clipping Algorithm:

For calculation of intersection point, first find the slope,

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

For bottom edge intersection point; $y = yw_{min}$

$$x = x_1 + \frac{y - y_1}{m}$$

For top edge intersection point; $y = yw_{max}$

$$x = x_1 + \frac{y - y_1}{m}$$

For left edge intersection point; $x = xw_{min}$

$$y = y_1 + m(x - x_1)$$

For right edge intersection point; $x = xw_{max}$

$$y = y_1 + m(x - x_1)$$

Numerical Section:

A region code is represented as TBRL with 0000 inside clipping window.
To calculate region code, perform following steps:

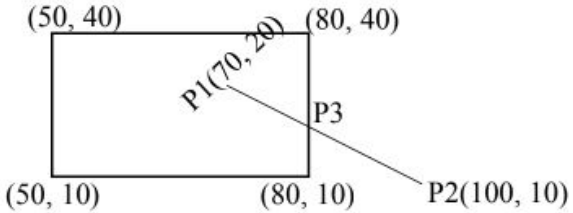
a) Calculate the difference between endpoint coordinates and clipping boundary i.e. $x - xw_{min}, xw_{max} - x, y - yw_{min}$ & $yw_{max} - y$.
b) Use '1' when resultant sign is $-ve$ otherwise use '0'.

X	Y	X & Y	X | Y
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

Q. Use the Cohen-Sutherland algorithm to clip the line P1(70, 20) and P2(100, 10) against a window lower left hand corner (50, 10) and upper right hand corner (80, 40).

Q. Use the Cohen-Sutherland algorithm to clip the line P1(70, 20) and P2(100, 10) against a window lower left hand corner (50, 10) and upper right hand corner (80, 40)

Solution:



Assign 4 bit binary code to the two end point

P1=0000

P2=0010

Finding bitwise OR:

P1|P2=0000|0010=0010

Since P1|P2 !=0000, hence the two point doesn't lie completely inside the window.

Finding bitwise AND:

P1&P2=0000 & 0010=0000

Since P1&P2=0000, hence line is partially visible.

Q. Use the Cohen-Sutherland algorithm to clip the line P1(70, 20) and P2(100, 10) against a window lower left hand corner (50, 10) and upper right hand corner (80, 40).

Now, finding the intersection of P1 and P2 with the boundary of window.

$p1(x_1, y_1) = (70, 20)$

$p2(x_2, y_2) = (100, 10)$

Slope $m = (10 - 20)/(100 - 70) = -1/3$

We have to find the intersection with right edge of window.

Here,

$x = 80$

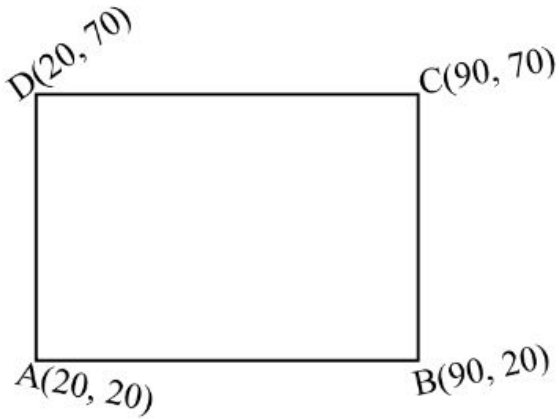
$$y = y_2 + m(x - x_2) = 10 + (-1/3)(80 - 100) = 10 + 6.67 = 16.67$$

Thus the intersection point P3 = (80, 16.67). So, discarding the line segment that lie outside the boundary i.e. P3P2, we get new line P1P3 with coordinate P1(70, 20) and P3(80, 16.67).

Q. Consider a rectangle clipping window with A(20, 20), B(90, 20), C(90, 70) & D(20, 70). Clip the line P1P2 with P1(10, 30) & P2(80, 90) using Cohen-Sutherland line clipping algorithm.

Q. Consider a rectangle clipping window with A(20, 20), B(90, 20), C(90, 70) & D(20, 70). Clip the line P1P2 with P1(10, 30) & P2(80, 90) using Cohen-Sutherland line clipping algorithm.

Solution:



Here,

$xw_{min} = 20, xw_{max} = 90$

$yw_{min} = 20, yw_{max} = 70$

Region code for $P1(10, 30)$:

$x - xw_{min} = 10 - 20 = -10$	1	L
$xw_{max} - x = 90 - 10 = 80$	0	R
$y - yw_{min} = 30 - 20 = 10$	0	B
$yw_{max} - y = 70 - 30 = 40$	0	T

Q. Consider a rectangle clipping window with A(20, 20), B(90, 20), C(90, 70) & D(20, 70). Clip the line P1P2 with P1(10, 30) & P2(80, 90) using Cohen-Sutherland line clipping algorithm.

Region code for $P2(80, 90)$:

$x - xw_{min} = 80 - 20 = 60$ 0 L

$xw_{max} - x = 90 - 80 = 10$ 0 R

$y - yw_{min} = 90 - 20 = 70$ 0 B

$yw_{max} - y = 70 - 90 = -20$ 0 T

∴ Region code for P1= 1000

∴ Region code for P2= 0001

- Check if line P1P2 is completely inside or outside.

Take OR of 0001 & 1000

0001
1000
1001 → Which is not equal to 0000 i.e. line P1P1 is not completely inside.

Take AND

0001
1000
0000 → Which means P1P2 is not completely outside.

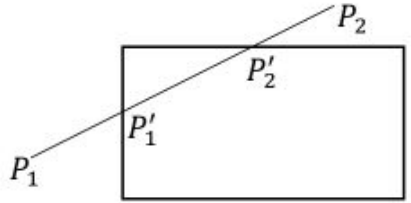
i.e. line P1P2 crosses the clipping window.

Q. Consider a rectangle clipping window with A(20, 20), B(90, 20), C(90, 70) & D(20, 70). Clip the line P1P2 with P1(10, 30) & P2(80, 90) using Cohen-Sutherland line clipping algorithm.

Now,

	T	B	R	L
P1:	0	0	0	1
P2:	1	0	0	0

The line crosses the top and left edge of the clipping window.



Now find P_1' & P_2'.

$$\text{Slope } m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{90-30}{80-10} = \frac{6}{7}$$

So for P_1'

$$x = 20$$

$$y = 30 + \frac{6}{7}(20 - 10) = 38.5$$

$$\therefore P_1' = (20, 38.5)$$

Q. Consider a rectangle clipping window with A(20, 20), B(90, 20), C(90, 70) & D(20, 70). Clip the line P1P2 with P1(10, 30) & P2(80, 90) using Cohen-Sutherland line clipping algorithm.

So for P_2'

$$y = 70$$

$$x = 10(90 - 60) \times \frac{7}{6} = 45$$

$$\therefore\ P_2' = (45, 70)$$

Thus the intersection point $P_1' = (20, 38.5)$ & $P_2' = (45, 70)$. So discarding the line segment that lie outside the boundary i.e. P_1P_1' & P_2P_2', we get new line $P_1'P_2'$ with coordinate $P_1' = (20, 38.5)$ & $P_2' = (45, 70)$.

Line Clipping Algorithm: Liang-Barsky Line Clipping Algorithm

3.4.2

Liang-Barsky Line Clipping Algorithm:

- This algorithm is considered to be the **faster parametric** line-clipping algorithm. The following **concepts are used** in this clipping:
  a. The parametric equation of the line.
  b. The **inequalities** describing the **range of the clipping window** which is used to determine the intersection between the line and the clip window.

The parametric equation of a line can be given by,

$$x = x_1 + u\Delta x$$

$$y = y_1 + u\Delta y, \qquad 0 \leq u \leq 1$$

Where, $\Delta x = x_2 - x_1$ & $\Delta y = y_2 - y_1$

Liang-Barsky Line Clipping Algorithm:

- The parameters p & q are defined as,
  - $p_1 = -\Delta x$, $q_1 = x_1 - xw_{min}$ (Left Boundary)
  - $p_2 = \Delta x$, $q_2 = xw_{max} - x_1$ (Right Boundary)
  - $p_3 = -\Delta y$, $q_3 = y_1 - yw_{min}$ (Bottom Boundary)
  - $p_4 = \Delta y$, $q_4 = yw_{max} - y_1$ (Top Boundary)
- **Conditions:**
- When the **line is parallel ($p_k = 0$)**to a view window boundary, the **p value** for the **boundary is zero**.
- When $p_k < 0$, as u increase line goes from the **outside to inside** (entering).
- When $p_k > 0$, line goes from the **inside to outside** (existing).
- When $p_k = 0$ & $q_k < 0$ then line is trivially invisible because it is **outside view window.**
- When $p_k = 0$ & $q_k > 0$ then line is **inside** the corresponding window boundary.

Liang-Barsky Line Clipping Algorithm:

Using the following conditions, the position of line can be determined:

Condition	Position of line
$p_k = 0$	Parallel to the clipping boundaries.
$p_k = 0$ & $q_k < 0$	Completely outside the boundary.
$p_k = 0$ & $q_k \geq 0$	Inside the parallel clipping boundary.
$p_k < 0$	Line proceeds from outside to inside.
$p_k > 0$	Line proceeds from inside to outside.

Parameters u_1 & u_2 can be calculated that define the part of line that lies within the clip rectangle. When,

1. $p_k < 0, maximum(0, \frac{q_k}{p_k})$ is taken.
2. $p_k > 0, minimum(0, \frac{q_k}{p_k})$ is taken.

If $u_1 > u_2$, the line is completely outside the clip window and it can be rejected. Otherwise, the endpoints of the clipped line are calculated from the two values of parameter u.

Liang-Barsky Line Clipping Algorithm:

Algorithm:

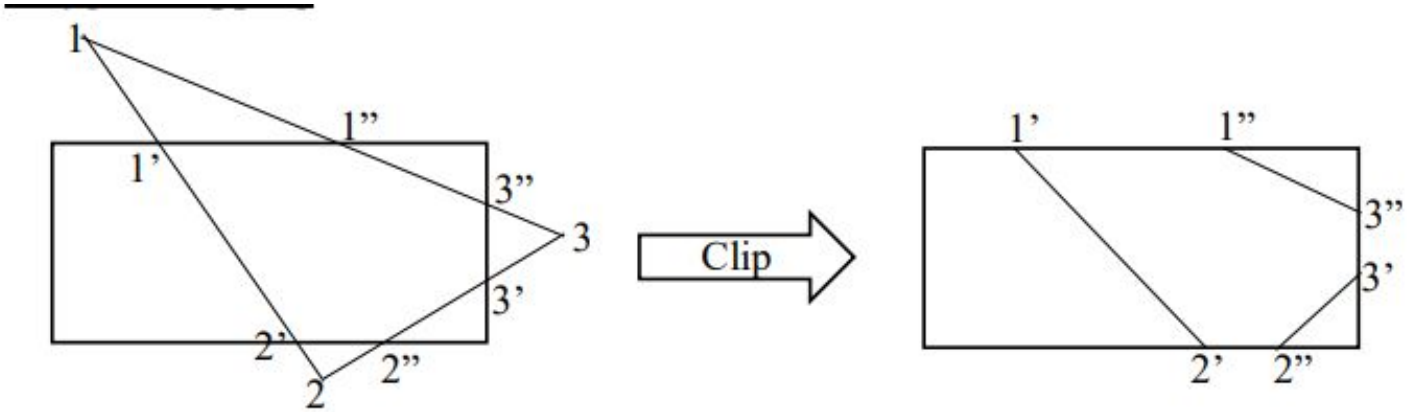
1. Set $u_{min} = 0, u_{max} = 1$
2. Calculate the values of u $(u_{left}, u_{right}, u_{top}, u_{bottom})$
   i. If $u < u_{min}$ or $u > u_{max}$ ignore that and move to the next edge.
   ii. Else separate the u values as entering or existing values using the inner product.
   iii. If u is entering value, set $u_{min} = u$; if u is existing value, set $u_{max} = u$.
3. If $u_{min} < u_{max}$, draw a line from $(x_1 + u_{min}(x_2 - x_1), y_1 + u_{min}(y_2 - y_1))$ to $(x_1 + u_{max}(x_2 - x_1), y_1 + u_{max}(y_2 - y_1))$.
4. If the line crosses over the window, $(x_1 + u_{min}(x_2 - x_1), y_1 + u_{min}(y_2 - y_1))$ and $(x_1 + u_{max}(x_2 - x_1), y_1 + u_{max}(y_2 - y_1))$ are the intersection point of line and edge.

Q. Apply Liang Barsky Line Clipping algorithm to the line with coordinates (30, 60) and (60, 25) against the window (xw_{min}, yw_{min}) = (10, 10) and (xw_{max}, yw_{max}) = (50, 50)

Polygon Clipping

Polygon Clipping:

Definition: Polygon clipping is the process of cutting or "clipping" a polygon so that it fits within a specified rectangular clipping window.



Sutherland-Hodgman Polygon Clipping Algorithm

3.4.3

Sutherland-Hodgman Polygon Clipping Algorithm:

- The Sutherland Hodgman algorithm is used for clipping polygons.
- This algorithm uses divide and conquer strategy.
- In this algorithm, all the vertices of the polygon are clipped successively against each edge of the given clipping window.
- It starts with the initial set of polygon vertices, first clips the polygon against the left boundary (edge) of the window.
- Then successively against the right boundary (edge), bottom boundary(edge) and finally against the top boundary(edge).

Sutherland-Hodgman Polygon Clipping Algorithm:

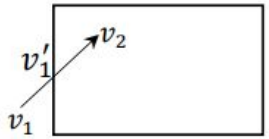
Four possible situations while processing :

1. If the first vertex is an outside the window, the second vertex is inside the window. Then second vertex is added to the output list. The point of intersection of window boundary and polygon side (edge) is also added to the output line.
2. If the first vertex is inside the window and second is an outside window. The edge which intersects with window is added to output list.
3. If both vertices are inside window boundary. Then only second vertex is added to the output list.
4. If both vertices are the outside window, then nothing is added to output list.

Sutherland-Hodgman Polygon Clipping Algorithm:

To find new sequence of vertices four cases are considered.

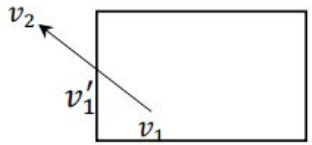
Case I



Movement: $out \rightarrow in$

Output: *intersection point, destination vertices i.e.*v_1' , v_2

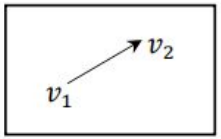
Case II



Movement: $in \rightarrow out$

Output: *intersection point i.e.* v_1'

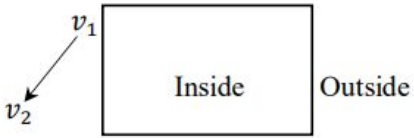
Case III



Movement: $in \rightarrow in$

Output: *destination vertices i.e.* v_2

Case IV



Movement: $out \rightarrow out$

Output: *none*

Sutherland-Hodgman Polygon Clipping Algorithm:

Four Cases of Polygon Clipping Against One Edge:

S.No	Case	Output
1	Wholly inside visible region (in-in)	Save endpoint
2	Exit visible region (in-out)	Save the intersection
3	Wholly outside visible region (out-out)	Save nothing
4	Enter visible region (out-in)	Save intersection and endpoint

Sutherland-Hodgman Polygon Clipping Algorithm:

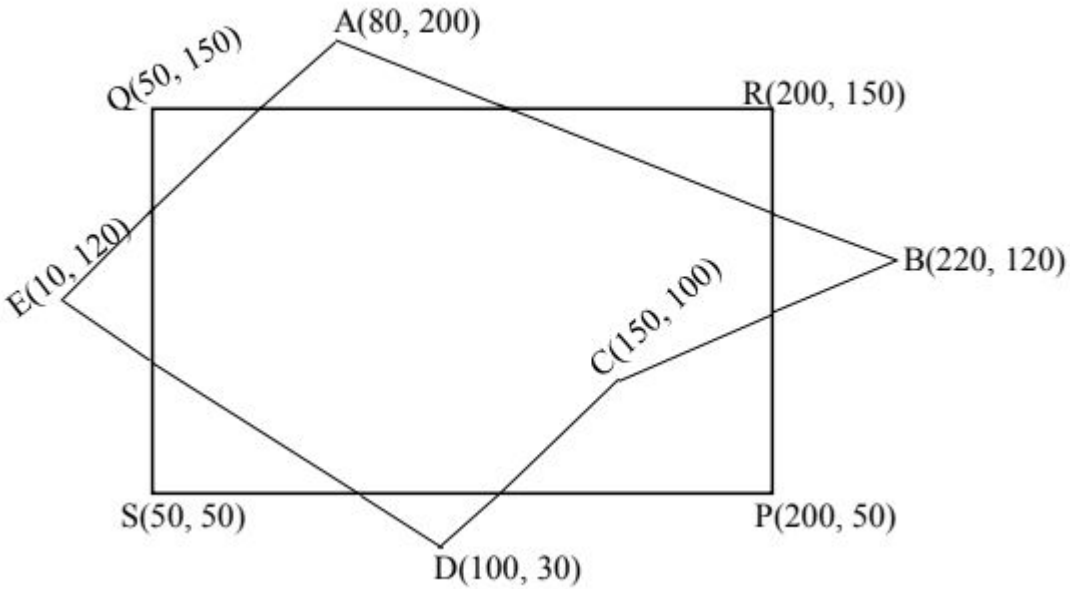
Algorithm :

For each clip window boundary perform following steps:

1. Construct an input vertex list (V0, V1 ..... Vn) where V0 = Vn.
2. Create empty output vertex list.
3. For each pair of adjacent vertices Vi and Vi+1 perform the following inside-outside test:
    a. If out-in (Vi is outside the window boundary and Vi+1 is inside),
        i. Add intersection point Vi' and Vi+1 to the output vertex list.
    b. If in-in (Both Vi and Vi+1 is inside the window boundary),
        i. Add Vi+1 to the output vertex list
    c. If in-out (Vi is inside the window boundary and Vi+1 is outside),
        i. Add intersection point Vi' to the output vertex list.
    d. If out-out (Both Vi and Vi+1 is outside the window boundary),
        i. Add nothing to output vertex list.
4. Stop

Q. Clip polygon against clipping window PQRS. The coordinates of polygon are A(80, 200), B(220, 120), C(150, 100), D(100, 30), E(10, 120). Coordinates of clipping window P(200, 50), Q(200, 150), R(50, 150) & S(50, 50).

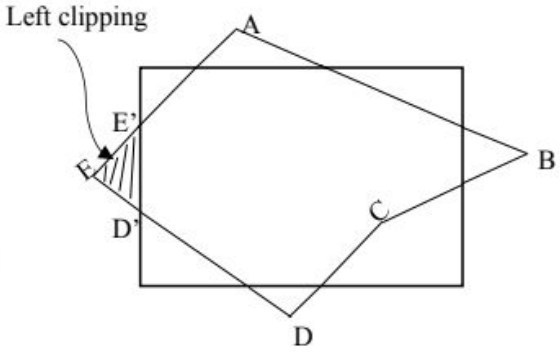
Q. Clip polygon against clipping window PQRS. The coordinates of polygon are A(80, 200), B(220, 120), C(150, 100), D(100, 30), E(10, 120). Coordinates of clipping window P(200, 50), Q(200, 150), R(50, 150) & S(50, 50).



Q. Clip polygon against clipping window PQRS. The coordinates of polygon are A(80, 200), B(220, 120), C(150, 100), D(100, 30), E(10, 120). Coordinates of clipping window P(200, 50), Q(200, 150), R(50, 150) & S(50, 50).

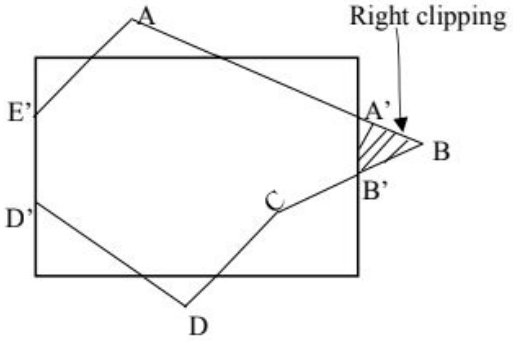
Left Clipping

vertex	case	O/P	
AB	*in* → *in*	B	
BC	*in* → *in*	C	
CD	*in* → *in*	D	
DE	*in* → *out*	D'	New vertex
EA	*out* → *in*	E', A	New vertex



Right Clipping

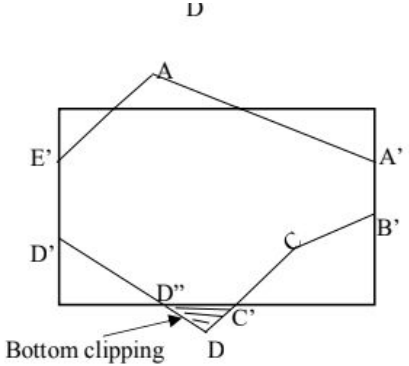
vertex	case	O/P	
AB	*in* → *out*	A'	New vertex
BC	*out* → *in*	B', C	New vertex
CD	*in* → *in*	D	
DD'	*in* → *in*	D'	
D'E'	*in* → *in*	E'	
E'A	*in* → *in*	A	



Q. Clip polygon against clipping window PQRS. The coordinates of polygon are A(80, 200), B(220, 120), C(150, 100), D(100, 30), E(10, 120). Coordinates of clipping window P(200, 50), Q(200, 150), R(50, 150) & S(50, 50).

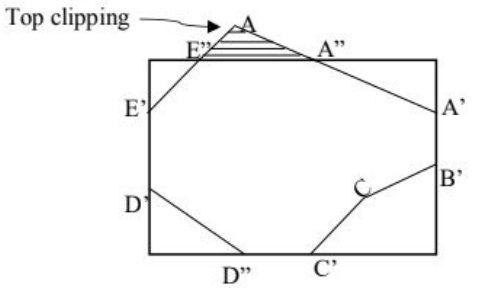
Bottom Clipping

vertex	case	O/P	
AA'	$in \rightarrow in$	A'	
A'B'	$in \rightarrow in$	B'	
B'C	$in \rightarrow in$	C	
CD	$in \rightarrow out$	C'	New vertex
DD'	$out \rightarrow in$	D", D'	New vertex
D'E'	$in \rightarrow in$	E'	
E'A	$in \rightarrow in$	A	



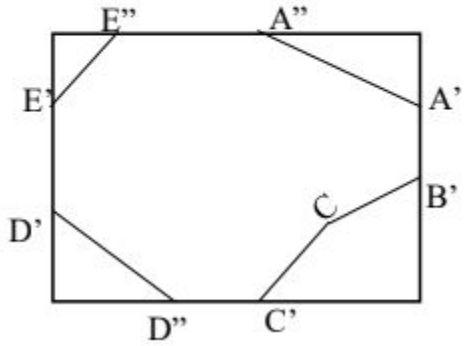
Top Clipping

vertex	case	O/P	
AA'	$out \rightarrow in$	A", A	New vertex
A'B'	$in \rightarrow in$	B'	
B'C	$in \rightarrow in$	C	
CC'	$in \rightarrow in$	C'	
C'D"	$in \rightarrow in$	D"	
D"D'	$in \rightarrow in$	D'	
D'E'	$in \rightarrow in$	E'	New vertex
E'A	$in \rightarrow out$	E"	New vertex



Q. Clip polygon against clipping window PQRS. The coordinates of polygon are A(80, 200), B(220, 120), C(150, 100), D(100, 30), E(10, 120). Coordinates of clipping window P(200, 50), Q(200, 150), R(50, 150) & S(50, 50).

Hence, final polygon after clipping:



THANKS!

Do you have any questions?

theciceerguy@p_m.me

9841893035

ghimires.com.np